

Object-Oriented Programming (OOP)

Pillar III & IV: Inheritance & Polymorphism

1. Pillar III: Inheritance (The Parent-Child Relationship)

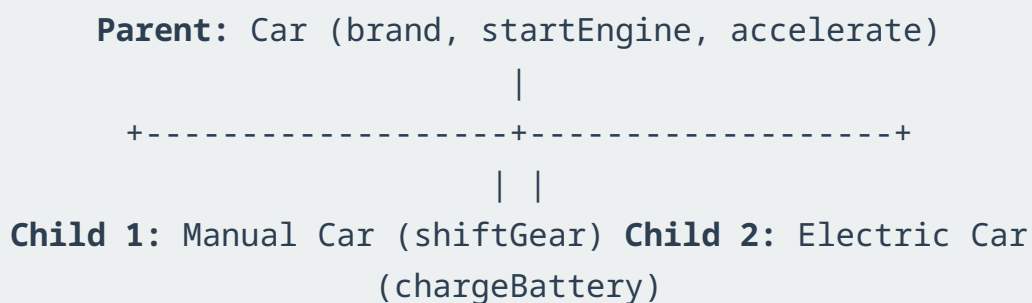
In the real world, objects are often related hierarchically. Inheritance allows us to map these real-world "parent-child" relationships directly into our code. It promotes code reusability by allowing a child class to absorb the characteristics (variables) and behaviors (methods) of a parent class.

Real-World Analogy: The Vehicle Hierarchy

Consider a generic **Car**. Every car has a brand, model, engine status, and speed. Every car can start, stop, accelerate, and brake. These are *generic properties*.

Now consider two specific types of cars:

- **Manual Car:** Inherits all generic properties from Car, but adds *specific* properties like `currentGear` and specific behaviors like `shiftGear()`.
- **Electric Car:** Inherits all generic properties from Car, but adds *specific* properties like `batteryPercentage` and specific behaviors like `chargeBattery()`.



Access Modifiers and Inheritance Modes

When inheriting a class, the access modifier dictates how the parent's members are treated inside the child class. The most widely used inheritance mode in system design is **Public Inheritance**.

Base Class Access Specifier	Public Inheritance	Protected Inheritance	Private Inheritance
Public (Accessible anywhere)	Stays Public in Child	Becomes Protected in Child	Becomes Private in Child
Protected (Accessible in Child classes)	Stays Protected in Child	Stays Protected in Child	Becomes Private in Child
Private (Accessible only in base)	Inaccessible in Child	Inaccessible in Child	Inaccessible in Child

2. Pillar IV: Polymorphism (Many Forms)

Polymorphism comes from Greek words: *Poly* (many) and *Morph* (forms). It means that objects or methods can take on multiple forms depending on the context.

In programming, polymorphism handles two main scenarios:

A. Dynamic Polymorphism (Method Overriding)

Scenario: Different objects from the same family react differently to the exact same instruction.

Analogy: If you tell a Duck, a Human, and a Tiger to "Run", they all execute the running behavior, but they each do it in an entirely different way.

Implementation: We define a generic method in the Parent class (e.g., `virtual void accelerate()` in `Car`). Then, the Child classes provide their own unique implementation:

- `ManualCar` overrides `accelerate()` to increase speed aggressively by 20 km/h.
- `ElectricCar` overrides `accelerate()` to increase speed gently by 15 km/h and simultaneously decreases the battery level.

This is resolved dynamically at runtime based on the actual object type being interacted with.

B. Static Polymorphism (Method Overloading)

Scenario: The exact same object changes its behavior based on different inputs/parameters provided to it.

Analogy: If you tell a human to "Run" in a relaxed setting, they jog. If you tell them to "Run" with a parameter `danger = true`, they sprint. The action is the same, but the parameters change the outcome.

Implementation: We create multiple functions in the exact same class with the *same name* but different parameters (signatures).

- `accelerate()`: Default acceleration.
- `accelerate(int targetSpeed)`: Accelerate according to the pressure applied to the pedal.

This is resolved statically at compile-time based on the arguments passed.

3. Summary: The 4 Pillars Working Together

A well-designed software system utilizes all four OOP pillars in harmony:

- **Encapsulation:** Groups variables and methods into a `Car` class and keeps sensitive data (like `currentSpeed`) private.
- **Abstraction:** Hides the complex logic of *how* acceleration works, simply exposing a clean `accelerate()` interface to the driver.
- **Inheritance:** Prevents writing duplicate code by allowing `ElectricCar` and `ManualCar` to inherit from the base `Car`.
- **Polymorphism:** Ensures that when we tell an `ElectricCar` to accelerate, it does so differently than a `ManualCar`, and it can adjust its behavior based on how hard the pedal is pressed.